VPN-LAN + Proxy — Open-Source Incorporation Survey

Revision: 1 **Last modified:** 2026-07-01T16:55:35Z **Status:** Research deliverable — deep multi-angle open-source survey (§11.4.150 + §11.4.8 + §11.4.99) for the VPN-LAN service-access feature on branch `feature/vpn-aware-dynamic-routing`. RESEARCH + DOC ONLY — no code written, nothing incorporated, no data-plane touched. **Authority:** Inherits `constitution/Constitution.md` per §11.4.35. Companion to the design set: `../../design/vpn_lan_access/PLAN.md`, `../../design/vpn_lan_access/architecture.md`, `../../design/vpn_lan_access/reflector_design.md`, and the current data-plane `config/dante/sockd.conf` + `config/squid/squid.conf`. **Scope note:** Every project claim carries a citation to the LATEST authoritative source (access date 2026-07-01, §11.4.99). Every deduction the sources do not directly state is marked **INFERENCE** (§11.4.6). §11.4.74 catalogue-check: none of these projects live in our own orgs (`vasic-digital` / `HelixDevelopment`), so each verdict is framed as *external reuse* — consumed as a deployed daemon or a rootless-Podman image booted via the containers submodule (§11.4.76), config-injected + decoupled (§11.4.28), never a git submodule of third-party code and never a project-specific hardcode. No recommendation here violates our constraints: rootless containers (§11.4.161), no CI/CD (§11.4.156), SSH-only git, decoupled submodules (§11.4.28).

---

0. Executive summary — top incorporations, ranked by leverage/risk

We already have a defensible core: an L3-routed VPN owned by `svord_toolkit`, a first-match Dante SOCKS ACL + Squid HTTP-CONNECT floor with an RFC1918/metadata SSRF block, and a standards-grounded reflector design. This survey looked for what would **materially strengthen or simplify** that, honestly (§11.4.6): where we already have the best fit, it says so.

#	Incorporation	What it buys us	Leverage	Risk	Adopt operagated
1	**smokescreen** (Stripe)	DNS-rebinding / TOCTOU-safe	HIGH	LOW (MIT, Go, single	**Adopt** (auton

#	Incorporation	What it buys us	Leverage	Risk	**Adopt opera gated**
		egress: re-resolves the hostname and re-checks the resolved IP against the allowlist *after* DNS, plus allowlist-by-default + report/enforce modes — closes a gap our Squid ACL floor does **not** cover for the HTTP-CONNECT path		daemon, local-stub testable like our existing SSRF teeth)	compl Squid not re the Da floor)
2	**alsmith/ multicast-relay**	Pins the open **INFERENCE** in `reflector_design.md` §3.2 — one daemon relays SSDP 1900 **and** mDNS 5353 (and Sonos 6969) across the interface boundary	MEDIUM-HIGH	LOW-MED (GPL-3.0 deployed-daemon → no linking contamination; but **LOCATION-rewrite is NOT documented** — a real residual gap)	Deplo **opera gated** (§11.4 remot tool-se + loca proof auton now
3	**toxiproxy** (Shopify)	Deterministic network-fault injection (latency / bandwidth / timeout / slicer / black-hole) as a TCP shim in front of every routed protocol — directly feeds §11.4.85 stress+chaos and §11.4.169 coverage, and is deterministic (§11.4.50)	HIGH (for the test mandate)	LOW (MIT, Go, API-driven, runs on loopback stubs — no live VPN needed)	**Adopt** (fully auton test to
4	**WireGuard mesh overlay** — Headscale or NetBird	GAME-CHANGER: collapses Phase 1 (routing) **and** Phase 12 (bidirectional	HIGH (architectural)	MED-HIGH (changes the VPN substrate that	**Opera gated** (§11.4 propo

#	Incorporation	What it buys us	Leverage	Risk	Adopt opera gated
		ingress allowlist) into one battle-tested both-way L3 control plane with per-host/-service ACLs, NAT traversal, and device posture — instead of hand-building return-routes + an ingress-allowlist		`svord_toolkit` owns)	not ad autono
5	**gvisor-tap-vsock / gvproxy** (containers org)	Rootless userspace network stack with dynamic port-forwarding over an HTTP API — a data-plane that forwards unicast without editing host route tables (§11.4.133-friendly) and is rootless-native (§11.4.161)	MEDIUM (INFERENCE — fit depends on topology)	MED (new moving part; userspace-netstack throughput ceiling)	Prototy worth the cr path

One-line verdict: incorporate **smokescreen** (harden egress) and **toxiproxy** (chaos coverage) now — both are low-risk, autonomous, and slot cleanly beside what we have; **pin multicast-relay** as the reflector tool (deploy operator-gated); and **surface the WireGuard mesh overlay to the operator** as the highest-leverage architectural simplification of the whole bidirectional design (§11.4.66), because it touches the `svord`-owned substrate and must not be adopted silently (§11.4.122).

---

1. Bidirectional L3 exposure over VPN / userspace networking

Our substrate is a `svord_toolkit`-owned WireGuard + L2TP/PPP L3 tunnel (`ppp0`, subnet `10.0.0.0/8`). PLAN §4.6 + Phase 12 require the exposure to work **both ways** with a default-deny ingress allowlist.

The core question: is there an off-the-shelf control plane that gives clean two-way routing + per-service ACLs so we do not hand-build return-routes and ingress plumbing?

Project	What it does	License	Maturity	Catalogue-check verdict	How into helix
Tailscale + Headscale	WireGuard mesh; **subnet routers** = L3 site-to-site (bridge two subnets both ways); HuJSON ACL policy; NAT traversal	Tailscale client BSD-3; **Headscale** control server BSD-3	Headscale v0.28.0 (2026-02-04), 38.5k★, very active	External reuse (self-hosted control plane, rootless-capable)	A sub route remo one o side g bidire `10/8`← routin policy — col PLAN 1 + P
NetBird	WireGuard overlay; groups + rules ACLs; **device posture checks**; self-hostable control plane; polished web UI	BSD-3 (except `management/,signal/,relay/` = AGPLv3), Go	26.6k★, active	External reuse	Same way - win a Head with check to ou allow hosts requi and a mana UI
Nebula (Slack)	Certificate-embedded group firewall (policy-as-code in the cert), no coordination-server ACLs, scales to tens of thousands	MIT, Go	Mature, widely deployed	External reuse	Cert- mode well t "allow X → s Y"; no UI, he cert-r ops
OpenZiti	App-layer zero-trust overlay — services advertise into a fabric, **no listening ports** on protected	Apache-2.0, Go + multi-language SDKs	Active, SDK-rich	External reuse (different model)	The " ports prope attrac the **in** surfa (Phas

Project	What it does	License	Maturity	Catalogue-check verdict	How into helix
	hosts, identity-authorized				a VPN reach expos proxy servic via autho ident an op
gvisor-tap-vsock / gvproxy	Pure-Go userspace netstack (gVisor), rootless, DNS + **dynamic port-forwarding via HTTP API**	Apache-2.0, Go	v0.8.x, maintained (containers org)	External reuse (rootless image via containers submodule)	Forwa unica from rootle conta **withe editi route** (§11.4 safe); dynar forwa fits ar ingre allow is progr
wireguard-go / BoringTun	Userspace WireGuard data-plane implementations (Go / Rust)	wireguard-go MIT; BoringTun BSD-3 (Cloudflare)	Mature	External reuse	Only if we need WireG *inside* conta rathe the k modu
sshuttle	TCP + DNS over SSH ("poor-man's VPN"); not L3; UDP weak	GPL-2.0, Python	Mature	No-match for our need	Could boots quick only a with z serve but **d route NFS-mult** and is bidire

Project	What it does	License	Maturity	Catalogue-check verdict	How into helix
					— str weak our e WG+ tunne

Honest finding (§11.4.6): for the *substrate itself* we already have a real L3 WireGuard tunnel via `svord_toolkit`, so sshuttle/wireguard-go/BoringTun are not upgrades. The genuine opportunity is the **control plane**: a mesh overlay (Headscale or NetBird) would replace the bespoke Phase-1 route-injection + Phase-12 return-route/ingress-allowlist plumbing with a battle-tested both-way router + policy engine. That is high leverage but **operator-gated** — it changes the `svord`-owned substrate and must be proposed via §11.4.66, never adopted silently (§11.4.122). **INFERENCE:** OpenZiti's "no listening ports" model is the single most interesting idea for our *ingress* attack surface (Phase 12), because it removes the open-port surface entirely instead of allowlisting it — worth a spike, but it is an app-layer rewrite, not a drop-in.

---

2. Multicast discovery reflection across L3

`reflector_design.md` already fixes Avahi (`enable-reflector=yes`) for mDNS and leaves an explicit **INFERENCE** open in §3.2: the exact SSDP/WS-Discovery relay tool (and the `LOCATION`-rewrite requirement) is unpinned. This angle pins it.

Project	What it does	License	Maturity	Catalogue-check verdict	How i in
Avahi (`enable-reflector=yes`)	Repeats mDNS `224.0.0.251:5353` across managed interfaces	LGPL-2.1, C	Standard Linux mDNS stack	External reuse (already in our design)	Alread Phase- reflect constr `allow-interf`
alsmith/multicast-relay	One daemon relays **SSDP 1900 + mDNS 5353 + Sonos 6969** between interfaces	**GPL-3.0**, Python (needs `netifaces`)	Actively used (Firewalla/UDM community), maintained	External reuse (deployed daemon on remote side)	**Pins t INFE** a singl coveri + mDN the ref

Project	What it does	License	Maturity	Catalogue-check verdict	How i in
					host ru tool in Avahi separa relay
smcroute	Static multicast routing between two interfaces (group forward, no app awareness)	GPL-2.0, C	Mature	External reuse (fallback)	Gener `239.25` forwar when advert URLs carry addres
igmpproxy	IGMP-based multicast forwarding (upstream/ downstream)	GPL-2.0, C	Mature	No-match (discovery)	IGMP memb forwar not SS mDNS relay
nberlee/ bonjour-reflector / **mdns-repeater**	VLAN-aware Bonjour reflectors	permissive (varies)	Community	External reuse (alt to Avahi)	Altern mDNS reflect Avahi' interfa model awkwa

Finding: multicast-relay resolves the design's open INFERENCE — it is the best single-tool fit for the "SSDP + mDNS on one reflector host" requirement, and it is what the Firewalla/ UniFi community actually deploys for Roku/Sonos/DIAL discovery. Two honest caveats to record before pinning it (§11.4.6): (a) **GPL-3.0** — acceptable because it is a standalone daemon booted via the containers submodule, not linked into helix_proxy; (b) the **`LOCATION`-rewrite** our reflector design demands (so the advertised URL is a routable `10.x`) is **not documented** in its README — we must verify its relay/proxy mode does this, or pair it with a small rewrite shim, before claiming the Cast/DIAL discovery path works end-to-end. Deployment stays operator-gated (§11.4.122). Update

`reflector_design.md` §3.2 from "INFERENCE, tool unpinned" to "candidate = multicast-relay, LOCATION-rewrite pending verification".

---

3. Reverse tunneling / ingress exposure

For Phase 12 ingress (VPN-host → an exposed proxy-side service). Honest framing first: if we adopt a **mesh overlay** (Angle 1 #4), bidirectional reach is native and a separate reverse-tunnel is **redundant**. Reverse tunnels matter only if we expose *one* proxy-side service to VPN hosts **without** a full overlay.

Project	What it does	License	Maturity	Catalogue-check verdict	How it slots in	R
frp (fast reverse proxy)	HTTP/ HTTPS/ TCP/UDP tunnels, subdomain routing, dashboard + Prometheus metrics	Apache-2.0, Go	~85k★, most feature-complete	External reuse (rootless image via containers submodule)	Expose a single proxy-side service inbound to VPN hosts with a client/server pair; metrics fit our observability mandate	A li s a (r t fl
rathole	High-perf NAT-traversal reverse proxy; tokens + Noise/TLS; hot-reload	Apache-2.0 (dual MIT), Rust	~12k★, active	External reuse	Same role as frp, lighter/ faster, mandatory service tokens (good default-deny posture)	F fe th si e
chisel	TCP/UDP over HTTP/ WebSocket; works through HTTP proxies	MIT, Go	Mature, popular	External reuse	Ingress that survives restrictive middle-boxes; single portable binary	W fr o
wstunnel (erebe)	TCP/UDP/ SOCKS5/ stdio over WS or HTTP2,	BSD-3, Rust	Active	External reuse	DPI/firewall-bypass ingress if the path is HTTP-only	C u m b

Project	What it does	License	Maturity	Catalogue-check verdict	How it slots in	R
	static binary					
bore	Minimal TCP tunnel	MIT, Rust	Popular, tiny	External reuse	Simplest possible single-port expose	M fe n la o

Finding (§11.4.6): for our design, a reverse tunnel is a **narrower** tool than what Phase 12 needs — the ingress requirement is *default-deny allowlist of (VPN-host → proxy-side-service) pairs*, which an overlay's policy engine (Angle 1) expresses natively and a point tunnel does not. If the operator declines the overlay, **rathole** (mandatory tokens, Noise/TLS, tiny footprint) or **frp** (metrics, maturity) is the cleanest single-service ingress — packaged rootless via the containers submodule, with its listening surface governed by the same default-deny allowlist + paired §1.1 teeth as the egress floor. **INFERENCE:** we likely do **not** need a reverse-tunnel at all if bidirectional overlay routing lands; keep this angle as a fallback, not a primary.

---

4. Proxy hardening / SSRF egress control

Our floor today is Dante first-match SOCKS ACL + Squid HTTP-CONNECT ACL with an RFC1918/link-local/loopback/metadata block. The question: is anything stronger than our current floor for the **egress carve-out** that Phase 1 opens?

Project	What it does	License	Maturity	Catalogue-check verdict	How it
smokescreen (Stripe)	HTTP CONNECT egress proxy: hostname ACL (allow/report/ enforce), **re-resolves DNS and re-checks the resolved IP** against internal/deny	MIT, Go	1.3k★, production at Stripe/ Fly/pretix, active	External reuse (rootless image via containers submodule)	Sits on **HTTP-CONNE egress** (beside behind for the carve-o the one Squid A — **post-re-chec**

Project	What it does	License	Maturity	Catalogue-check verdict	How it
	ranges, blocks non-routable IPs, mTLS client auth				defeats rebindi TOCTO a hostna resolves internal the ACL allowlis default mode fo rollout
OpenBSD relayd	App-layer gateway / transparent proxy / load-balancer with protocol filters	ISC, C	Very mature, OpenBSD base	External reuse	A harde transpa proxy o we ever Squid
Envoy	Universal L4/ L7 data plane, rich egress filtering, programmable	Apache-2.0, C++	Mature, heavy	External reuse	Powerfu policy (ext-auth need a s mesh-gr plane
HAProxy	High-perf TCP/HTTP proxy, ACLs, ~10-20% less CPU than Envoy for HTTP	GPL-2.0/ LGPL, C	Battle-tested (15+ yr)	External reuse	Efficien ACL fro we outg Squid
Squid ACL best-practice	Our current HTTP floor	GPL-2.0, C	In place	Already used	Keep — RFC191 metadat `SSL_Port` `extensio` work is

Finding: our Dante + Squid floor is a **sound baseline** — but **smokescreen adds a genuinely stronger guarantee on the HTTP-CONNECT path**: it re-resolves the hostname and re-validates the *resolved IP* against the deny ranges after DNS, which our static Squid ACL does not do, closing the DNS-rebinding / TOCTOU sub-class of SSRF (the exact class OWASP SSRF Case-1 warns about, which PLAN §4 already references). It

is MIT, single-binary Go, and — crucially — **testable exactly like our existing SSRF teeth** (`report`/`enforce` modes + local-stub targets), so it lands autonomously with a paired §1.1 mutation proving an out-of-allowlist resolved-IP still denies. Honest boundary (§11.4.6): smokescreen helps **only** the HTTP(S) egress path; the L3-routed protocols (SMB/NFS/FTP/mail/ADB) are not HTTP and stay governed by the Dante first-match ACL + a host-level firewall (nftables) — smokescreen is a **complement**, not a replacement for the whole floor. relayd/Envoy/HAProxy are viable if we ever replace Squid, but none is a compelling reason to churn a working floor now.

---

5. Anti-bluff / test-oracle & discovery tooling

For §11.4.85 stress+chaos and §11.4.169 comprehensive coverage of the routed data-plane, plus discovery oracles.

Project	What it does	License	Maturity	Catalogue-check verdict	How it slots i
toxiproxy (Shopify)	TCP proxy with deterministic **toxics** (latency, bandwidth, timeout, slicer, slow_close, black-hole, limit_data), API-driven	MIT, Go	Widely used, active	External reuse (test tooling)	A shim in fron any routed pro (SMB/NFS/FT mail/ADB/Cast control) to inje faults deterministica §11.4.85 chaos §11.4.50 determinism; on **loopback stubs, no live** → fully autono
pumba	Container chaos: kill/ stop/pause + `netem` delay/ loss/corrupt; **supports Podman**	Apache-2.0, Go	Active	External reuse (rootless via containers submodule)	Inject containe level network (delay/loss) ag our reflector/a server contain §11.4.85 proce death + netwo fault classes o rootless Podm (§11.4.161)
blockade	Docker network partitions / splits for	Apache-2.0, Python	Older, less active	External reuse (niche)	Partition-testin we run a multi container

Project	What it does	License	Maturity	Catalogue-check verdict	How it slots i
	distributed-failure testing				reflector+adb topology
avahi-browse / gssdp-discover / nmap	Discovery + scan oracles	LGPL/GPL	Standard	Already implied in tests	The read-side oracle our `discovery_refl` already uses; for routed-por reachability evidence

Finding: toxiproxy is the strongest single incorporation for the test mandate — deterministic, API-driven, loopback-capable, and it directly satisfies §11.4.85's latency/bandwidth/black-hole classes on our routed TCP paths **without needing the live VPN**, so it lands on the autonomous slate today with paired §1.1 mutations proving each toxic actually degrades the path (an analyzer that PASSes with the toxic active is a bluff gate, §11.4.107(10)). **pumba** complements it for container-level chaos against the reflector/adb-server images on rootless Podman. Together they close the §11.4.85 + §11.4.169 chaos/coverage requirement with real captured evidence rather than a happy-path-only suite.

---

6. Game-changing approaches

Two ideas would materially change the shape of the whole design, and one resolves an open design gap.

A. WireGuard mesh overlay as the bidirectional substrate (GAME-CHANGER, operator-gated). The single biggest simplification: instead of hand-building Phase-1 route injection + Phase-12 return-routes + an ingress-allowlist state machine, adopt a self-hosted overlay control plane (**Headscale** BSD-3 or **NetBird** BSD-3/AGPLv3) whose *subnet-router* feature is exactly bidirectional L3 site-to-site, and whose policy engine expresses "allow VPN-host X → proxy-side service Y" natively — the precise Phase-12 requirement — with NAT traversal and device posture as bonuses. This would fold two of our security-critical phases into one battle-tested, widely-audited component. **The catch (§11.4.6/ §11.4.122):** it changes the VPN substrate that `svord_toolkit` owns, and PLAN §1 hard-constraint 2 forbids changing `svord/` remote hosts without an interactive operator decision. So this is a **proposal to surface via §11.4.66**, not an autonomous adoption. It is the highest-leverage architectural idea in this survey.

B. Rootless userspace data-plane (gvisor-tap-vsock / gvproxy) (INFERENCE — spike-worthy). A pure-Go userspace netstack with a dynamic port-forwarding HTTP API lets a rootless container forward unicast **without touching host route tables** — which aligns with §11.4.133 (never mutate host/target routing autonomously) and §11.4.161 (rootless). **INFERENCE:** whether its throughput ceiling suits bulk SMB/NFS transfer is unproven and needs a benchmark; but for the *control/ingress* plane (programmatic port-forward = programmatic ingress-allowlist) it is an elegant fit worth a spike.

C. Pin the reflector tool (resolves an open design INFERENCE). `reflector_design.md` §3.2 leaves the SSDP relay unpinned; **multicast-relay** (Angle 2) is the community-proven single-daemon answer for SSDP + mDNS, closing that gap the moment its `LOCATION`-rewrite behaviour is verified. Not architecture-changing, but it converts an open design question into a decided one.

Non-game-changers, stated honestly (§11.4.6): we do **not** need sshuttle (weaker than our WG tunnel), we do **not** need a reverse-tunnel if the overlay lands (redundant), and we should **not** churn the working Squid/Dante floor for Envoy/HAProxy without a driving requirement. The floor is sound; smokescreen sharpens its weakest sub-class (HTTP-CONNECT DNS-rebinding), and that is the only egress change worth making now.

---

7. Recommended next steps (what to prototype first + why)

1. **Prototype smokescreen on the HTTP-CONNECT egress path (autonomous, now).** Boot it as a rootless-Podman image via the containers submodule (§11.4.76), config-injected with the `HELIX_BRIDGE_SUBNET` allowlist. Prove the DNS-rebinding re-check with a local-stub target that resolves to an internal IP → `enforce` denies; ship a paired §1.1 mutation (an out-of-allowlist resolved IP still denies). This mirrors our existing `ssrf_carveout_teeth.sh` discipline and needs no live VPN. **Why first:** highest leverage-to-risk, closes a real SSRF sub-class, fully autonomous.
2. **Wire toxiproxy into the §11.4.85 chaos layer (autonomous, now).** Put a toxiproxy shim in front of each routed protocol's *local stub* and assert every toxic (latency/black-hole/timeout) genuinely degrades the path, with a paired §1.1 mutation so the analyzer cannot bluff. **Why:** satisfies the stress+chaos + determinism mandate today without the operator-gated live paths.
3. **Verify multicast-relay's `LOCATION`-rewrite, then pin it in `reflector_design.md` §3.2.** Stand it up against a local SSDP/mDNS

stub; confirm whether it rewrites `LOCATION` to a routable address or needs a shim. Update the design doc from INFERENCE to a decided tool. **Why:** converts an open design gap into a fact; deploy stays operator-gated.
4. **Surface the WireGuard mesh overlay to the operator (§11.4.66).** Present Headscale vs NetBird as options with the blast radius (it replaces `svord`-owned routing + the Phase-12 ingress plumbing) and the trade-off, and let the operator decide keep-current vs adopt-overlay. **Why:** it is the biggest architectural simplification but it touches the substrate we are forbidden to change silently (§11.4.122).
5. **(Optional) Spike gvproxy for the rootless forward/ingress plane** with a throughput benchmark vs the kernel-routed path. **Why:** promising for §11.4.133-safe routing, but unproven for bulk transfer — measure before betting.

Nothing above is incorporated by this document; each is a scoped, evidence-first prototype that would cross independent review (§11.4.142) + the §11.4.169 test matrix before landing, and each respects rootless-containers (§11.4.161), no-CI (§11.4.156), SSH-only git, and decoupled-submodule (§11.4.28) constraints.

Sources verified 2026-07-01

- Headscale (self-hosted Tailscale control server, BSD-3, v0.28.0): https://github.com/juanfont/headscale
- Tailscale subnet routers (L3 site-to-site): https://tailscale.com/docs/features/subnet-routers · https://tailscale.com/docs/features/site-to-site · https://headscale.net/stable/ref/routes/
- Tailscale open source: https://tailscale.com/opensource
- NetBird (WireGuard overlay, BSD-3 + AGPLv3 for management/signal/relay, ACLs, posture checks): https://github.com/netbirdio/netbird · https://netbird.io/knowledge-hub/top-5-opensource-alternatives-to-tailscale2
- Nebula (Slack overlay, MIT, cert-group firewall): https://github.com/slackhq/nebula (via https://netbird.io/knowledge-hub/top-5-tailscale-alternatives)
- OpenZiti (Apache-2.0, app-layer zero-trust, no listening ports): https://github.com/openziti/ziti · https://github.com/openziti/ziti/blob/main/README.md
- gvisor-tap-vsock / gvproxy (Apache-2.0, userspace netstack, dynamic port-forward): https://github.com/containers/gvisor-tap-vsock · https://github.com/containers/gvisor-tap-vsock/blob/main/README.md · https://deepwiki.com/containers/gvisor-tap-vsock/2.1-gvproxy
- gVisor networking / netstack: https://gvisor.dev/docs/user_guide/networking/

- wireguard-go (MIT userspace WireGuard): https://github.com/WireGuard/wireguard-go (via https://www.saashub.com/compare-wireguard-vs-sshuttle)
- BoringTun (Cloudflare, BSD-3 userspace WireGuard): https://github.com/cloudflare/boringtun (via https://pinggy.io/blog/top_open_source_tailscale_alternatives/)
- sshuttle (GPL-2.0, TCP+DNS over SSH, not L3): https://github.com/sshuttle/sshuttle
- Avahi mDNS reflector (`enable-reflector=yes`, LGPL): https://linux.die.net/man/5/avahi-daemon.conf · https://www.avahi.org/
- alsmith/multicast-relay (GPL-3.0, SSDP 1900 + mDNS 5353 + Sonos 6969): https://github.com/alsmith/multicast-relay · https://github.com/alsmith/multicast-relay/blob/master/README.md · https://github.com/alsmith/multicast-relay/issues/56
- SSDP across subnets / reflector patterns: https://twisteroidambassador.github.io/2025/03/20/ssdp-across-subnets.html · https://forum.mikrotik.com/t/the-complete-ssdp-mdns-solution-for-network-segmentation/167825 · https://blog.christophersmart.com/2020/03/30/resolving-mdns-across-vlans-with-avahi-on-openwrt/
- smcroute (GPL-2.0, static multicast routing): https://github.com/troglobit/smcroute
- frp (Apache-2.0, Go, feature-complete reverse proxy): https://github.com/fatedier/frp (via https://xtom.com/blog/frp-rathole-ngrok-comparison-best-reverse-tunneling-solution/)
- rathole (Apache-2.0/MIT, Rust, high-perf NAT traversal): https://github.com/rathole-org/rathole
- chisel (MIT, TCP/UDP over WebSocket): https://github.com/jpillora/chisel (via https://github.com/anderspitman/awesome-tunneling)
- wstunnel (erebe, BSD-3, Rust, WS/HTTP2 tunneling): https://github.com/erebe/wstunnel
- bore (MIT, Rust, minimal tunnel): https://github.com/ekzhang/bore (via https://pinggy.io/blog/best_ngrok_alternatives/)
- awesome-tunneling (self-hosting tunnel catalogue): https://github.com/anderspitman/awesome-tunneling
- smokescreen (Stripe, MIT, Go, SSRF egress proxy with post-DNS IP re-check): https://github.com/stripe/smokescreen · https://github.com/stripe/smokescreen/blob/master/README.md · https://fly.io/blog/practical-smokescreen-sanitizing-your-outbound-web-requests/
- OpenBSD relayd (ISC, app-layer gateway/transparent proxy): https://man.openbsd.org/relayd.conf.5 · https://man.openbsd.org/relayd.8
- Envoy vs HAProxy (egress/proxy comparison): https://last9.io/blog/envoy-vs-haproxy/
- SSRF prevention background (OWASP-class, DNS-rebinding): https://goteleport.com/blog/ssrf-attacks/
- toxiproxy (Shopify, MIT, deterministic TCP toxics): https://github.com/Shopify/toxiproxy · https://chaostoolkit.org/drivers/toxiproxy/

- pumba (Apache-2.0, Go, container chaos + netem, Podman support): https://github.com/alexei-led/pumba
- blockade (Apache-2.0, Python, Docker network partitions): https://github.com/exajobs/chaos-engineering-collection (referenced)
- Chaos-engineering tool landscape (2025): https://steadybit.com/blog/top-chaos-engineering-tools-worth-knowing-about-2025-guide/

Access date for all sources: 2026-07-01 (§11.4.99). FACT items (licenses, star counts, protocol/port facts, feature descriptions) are grounded in the cited sources as fetched on the access date; every item marked ***INFERENCE*** *(§11.4.6) — overlay-substrate fit for our exact* `svord` *topology, gvproxy throughput suitability for bulk transfer, reverse-tunnel redundancy under an overlay, and multicast-relay's undocumented* `LOCATION`*-rewrite behaviour — is a deduction to be proven by a scoped prototype before it is asserted as settled, never claimed here as decided. Deep-research (§11.4.150), multi-angle: substrate/control-plane (Angle 1), discovery-relay (Angle 2), ingress-tunnel (Angle 3), egress-hardening (Angle 4), chaos/oracle (Angle 5), architectural (Angle 6).*